DATA ENGINEERING & CLOUD · LESSON 15 OF 15

Technical Documentation, Runbooks and Production Troubleshooting for Pipelines and AI Apps

Write the documentation that keeps a data platform running when its authors are away: a README, a data dictionary and runbooks for common incidents. Then troubleshoot a live problem in a fixed order, from mitigation to root cause, and finish with a short post-incident review that adds a test or alert.

Gap-fill lesson · 2026 · Intermediate · 120 min

By the end of this lesson you can

- Write a README, a data dictionary entry and a runbook for a pipeline
- Mitigate a live problem before diagnosing its cause
- Follow a repeatable troubleshooting sequence for pipelines and AI applications
- Write a post-incident review that ends in a test, an alert or a control

Datasets used

`daily_sales.csv`

Documentation is part of the system

A pipeline that only its author understands is a liability. When that person is on leave, a small failure becomes a long outage. Documentation is what lets someone else act on a problem at night, and it is cheaper to write while the system is fresh. Three documents do most of the work: a README, a data dictionary, and runbooks.

CASE STUDY **The dashboard shows yesterday's numbers**

At eight in the morning, a manager reports that the dashboard is a day behind. The engineer who built the pipeline is travelling. The runbook for "dashboard shows yesterday's numbers" lists four checks in order, names the owner of the source file, and says whom to call if the file is late. The problem is resolved by a colleague in twenty minutes.

The README

The README is the first page anyone reads. It should answer five questions in under a page:

1. **What does this do?** One paragraph, with the business purpose.
2. **What does it read and write?** Sources, destinations and refresh times.
3. **How do I run it?** Setup and the exact commands.
4. **Who owns it?** Team, named owners and how to reach them.
5. **What can go wrong?** Links to the runbooks.

MARKDOWN

```
# Sales daily load

Loads yesterday's point-of-sale files into the reporting warehouse and rebuilds the sales
dashboard tables.

- Reads: /data/landing/pos/*.csv (delivered by 05:30 by the store systems team)
- Writes: warehouse schema `gold`, tables `daily_sales`, `monthly_sales`
- Schedule: 06:00 daily (Airflow DAG `nightly_sales_close`)
- Owner: data-engineering (primary: Asha; backup: Ravi)
- Runbooks: docs/runbooks/dashboard-stale.md, docs/runbooks/load-failed.md
```

The data dictionary

A data dictionary describes every table and column: what it means, its type, its allowed values, its owner and its sensitivity. It is the document analysts reach for when a number looks strange.

Table	Column	Type	Meaning	Allowed values	Classification
daily_sales	date	date	Calendar day of the sale	Any day since 2022-01-01	Internal
daily_sales	units_sold	integer	Units sold on that day across all stores	0 or more	Internal
customers	customer_id	text	Stable customer key	Format C followed by four digits	Personal (pseudonymised in gold)

Write the meaning from the business point of view, not from the database type. "Units sold, net of returns" is a meaning; "integer" is only a type.

Runbooks

A **runbook** is a step-by-step guide for a known incident. Each runbook has the same structure, so that the reader can act under pressure:

Section	Content
Symptom	What the user or the alert sees
Impact	Who is affected and what decisions are at risk
Checks	Ordered steps, each with the command or the screen and the expected result
Fixes	The safe actions for each finding
Escalation	Who to call and when
Verification	How to confirm the problem is solved

MARKDOWN

```
# Runbook: dashboard shows yesterday's numbers

Symptom: the sales dashboard's "last refreshed" time is before 06:30.
Impact: managers see yesterday's sales; morning decisions are delayed.

Checks
1. Open the Airflow run for today. Expected: nightly_sales_close is success or running.
2. If failed, open the first failed task log. Note the error.
3. Check the landing folder: ls -lh /data/landing/pos/. Expected: one file per store,
today's date.

Fixes
- Files missing: contact the store systems team (see escalation). Do not rerun until files
arrive.
- Task failed with a transient error: clear the failed task and rerun it. The load is
idempotent.
- Quality check failed: do not override. Escalate to the data owner.

Escalation: data-engineering on-call at 07:00; the data owner after 08:00 if still
unresolved.
Verification: dashboard refresh time is after 06:30 and the row count matches the file
count.
```

::::tip Write the runbook from the last incident The best runbooks come from real incidents. After each one, add the steps that were discovered during it, and remove steps that turned out to be wrong. ::::

Production troubleshooting: the order matters

When a live system fails, people often start by looking for the cause. The better order is to limit the damage first, then find the cause, then fix it properly.

Step	Action	Why this order
1. Confirm the impact	Who is affected, and since when	Decides how urgent the response is
2. Mitigate	Rerun, roll back, switch to a safe source, or disable the feature	Stops harm while the cause is unknown
3. Check recent changes	Deployments, schema changes, new data sources, config changes	Most failures follow a change
4. Check logs and freshness	The first error, upstream freshness, row counts	Locates the failing component
5. Reproduce	Run the failing step on a copy or on a small input	Confirms the cause before a fix
6. Fix the root cause	The smallest change that removes the cause	Prevents recurrence
7. Verify	The original symptom is gone, and the checks pass	Proves the fix
8. Prevent	Add a test, an alert or a schema check	Makes the next occurrence visible early

CASE STUDY **A column that was renamed upstream**

A dashboard showed blank revenue for one region. Step 1 confirmed one region; step 2 switched the dashboard to the previous day's gold table; step 3 found that the source team had renamed a column the day before; step 4 showed the load succeeded with nulls in that column; step 5 reproduced it on a sample; step 6 mapped the new name; step 7 verified the numbers against the ledger; step 8 added a schema-change alert. Each step was short, and the sequence was the reason the incident ended quickly.

Troubleshooting an AI application

AI applications fail in new ways, so add these checks to the sequence:

- **Retrieval.** Was the right document retrieved? Look at the retrieved chunks for the failing question.

- **Model input.** Did the prompt contain what was expected (the right context, no leaked secrets, the right version)?
- **Tools.** Did each tool return a valid result? A tool error often looks like a model error.
- **Version changes.** Did the prompt, the model or the index change? Compare the regression suite with the last run.
- **Cost and latency.** A sudden rise often means a loop or a larger context than intended.

The post-incident review

A post-incident review is short and blameless. It has five parts: the timeline, the impact, the root cause, what worked and what did not, and the actions. Each action has an owner and a date.

MARKDOWN

```
# Post-incident review: revenue blank for one region (2026-11-03)

Timeline: 06:40 alert; 07:05 impact confirmed; 07:20 dashboard switched to yesterday; 09:10
fix deployed.
Impact: one region showed blank revenue for 2.5 hours; two managers used the previous day's
numbers.
Root cause: upstream renamed a column without notice; the load succeeded with nulls.
What worked: the rollback to the previous gold table; the runbook step that checked
upstream row counts.
What did not: no schema check on the source; the alert fired on the dashboard, not on the
load.
Actions:
- Add a schema-change check to the load (owner: Ravi, due 2026-11-20).
- Require notice for source schema changes in the data contract (owner: Asha, due 2026-11-
30).
```

::::warn Blameless does not mean vague "Blameless" means the review looks at the system, not at a person. It must still name the specific cause and the specific control. "Communication could be better" is not an action; "the source team posts schema changes in the channel 48 hours ahead" is. ::::

INTERVIEW CORNER

How do you troubleshoot a pipeline that is producing wrong numbers?

Confirm the impact and limit the damage first, by switching to the last good data if necessary. Check recent changes, the logs and the freshness of upstream data, and reproduce the problem on a small input. Fix the cause, verify against a trusted total, and add a test or alert so the same error is caught next time. Then write a short review with an owner for each action.

::::key What to remember - Good documentation is a README, a data dictionary and runbooks with checks, fixes and escalation. - Limit the damage first, then find the cause, then fix, verify and prevent. - Most failures follow a change; check recent changes early. - For AI applications, check retrieval, model input, tools, version changes and cost. - A post-incident review is blameless and ends with owned, dated actions. ::::

Quick quiz

5 questions · tap an answer to check it

1. What is the first priority in production troubleshooting?

A Find the person who made the change

B Confirm the impact and mitigate the harm

C Write the post-incident review

D Rebuild the whole platform

Answer: B. Stopping the harm comes before finding the cause.

2. What should a runbook contain?

A Only the author's name

B Symptom, impact, ordered checks, fixes, escalation and verification

C A list of colours

D The full source code

Answer: B. Runbooks are used under pressure, so each section must be actionable.

3. Why is a data dictionary useful?

A It makes queries faster

B It explains what each column means, its allowed values and its owner

C It replaces the database

D It stores passwords

Answer: B. Dictionaries answer "what does this number mean?"

4. In the example, what caused the blank revenue?

A A slow query

B A column renamed upstream, so the load succeeded with nulls

C A broken chart

D A missing user

Answer: B. The root cause was a silent upstream schema change.

5. Which action best prevents a repeat of a silent schema change?

A Read the logs more carefully

B Add a schema-change check that alerts before the load publishes

C Retrain the team

D Remove the dashboard

Answer: B. A preventive control makes the failure visible early.

EXERCISE **Write a runbook and a review**

Write a runbook for one failure in a pipeline you know, with the symptom, impact, five ordered checks, the safe fixes and the escalation path. Then write a post-incident review for an invented incident that uses the runbook, with one owned, dated action that is a test or an alert.

Show solution (worked answer)

Hints: each check needs an expected result, so that the reader knows when to move on. The action must be something you can test: a schema comparison, a freshness alert, or a reconciliation.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.